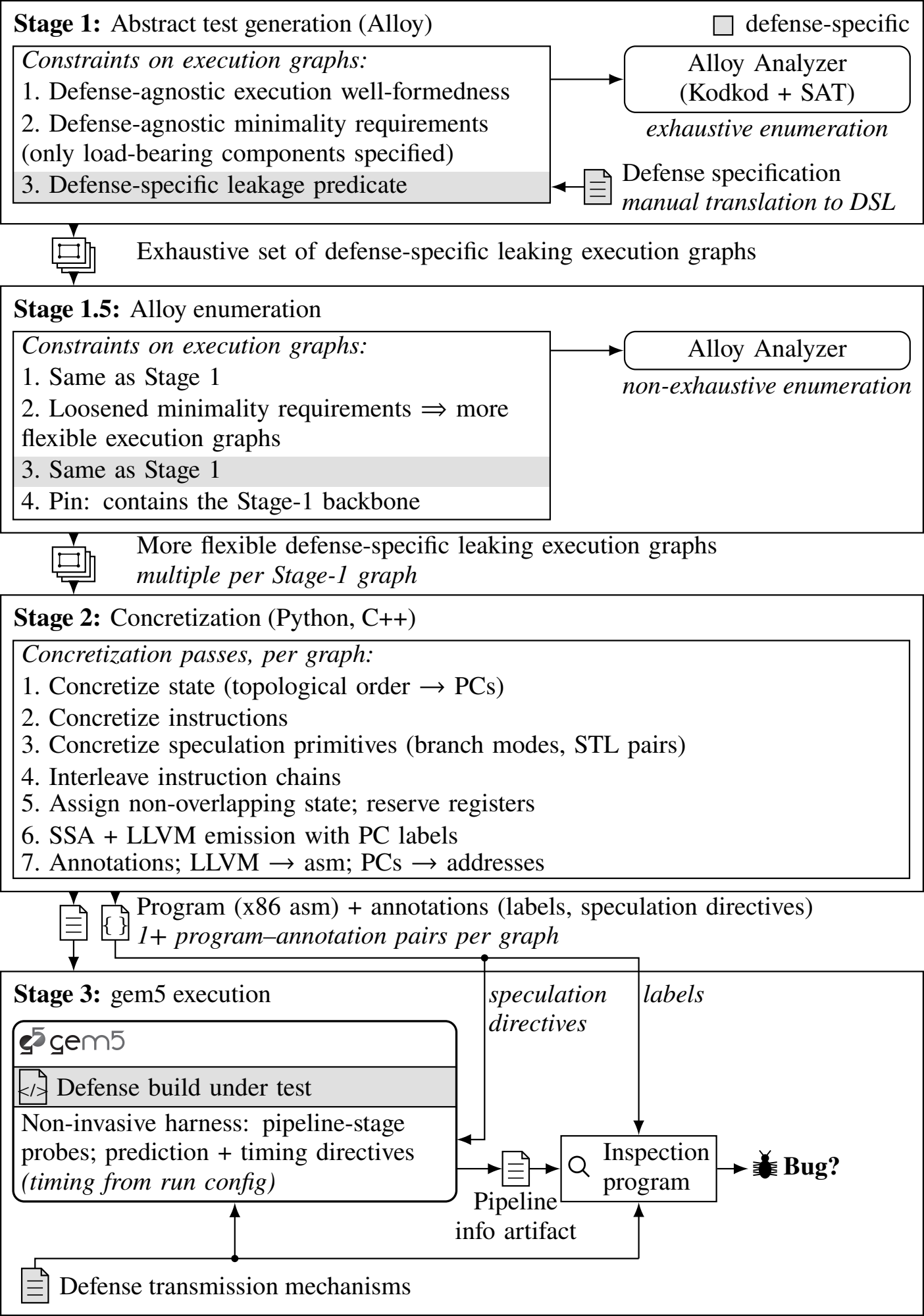

Stage 1: Abstract test generation (Alloy)
defense-specific
Constraints on execution graphs:
1. Defense-agnostic execution well-formedness
2. Defense-agnostic minimality requirements (only load-bearing components specified)
3. Defense-specific leakage predicate
Alloy Analyzer (Kodkod + SAT)
exhaustive enumeration
Defense specification
manual translation to DSL
Exhaustive set of defense-specific leaking execution graphs
Stage 1.5: Alloy enumeration
Constraints on execution graphs:
1. Same as Stage 1
2. Loosened minimality requirements ⇒ more flexible execution graphs
3. Same as Stage 1
4. Pin: contains the Stage-1 backbone
Alloy Analyzer
non-exhaustive enumeration
More flexible defense-specific leaking execution graphs
multiple per Stage-1 graph
Stage 2: Concretization (Python, C++)
Concretization passes, per graph:
1. Concretize state (topological order → PCs)
2. Concretize instructions
3. Concretize speculation primitives (branch modes, STL pairs)
4. Interleave instruction chains
5. Assign non-overlapping state; reserve registers
6. SSA + LLVM emission with PC labels
7. Annotations; LLVM → asm; PCs → addresses
Program (x86 asm) + annotations (labels, speculation directives)
1+ program–annotation pairs per graph
Stage 3: gem5 execution
speculation directives
labels
gem5
Defense build under test
Non-invasive harness: pipeline-stage probes; prediction + timing directives
(timing from run config)
Pipeline info artifact
Inspection program
Bug?
Defense transmission mechanisms